the pipeline. The documentation here provides details on the various options within this section; let's have a brief look at the options available for Docker.

Option 1: For simple use cases that involve the pipeline (a node on which a build job runs as per the label), this suffices – a Docker image serves as the build agent (a.k.a. node, FKA or slave).

```
pipeline {
  agent {
      docker {
          image 'name-of-image'
          label 'preconfigured-node-to-download-this-
image'
      }
  }
}
```

Option 2: The pipeline will execute the stage(s) using the container built from Dockerfile, located in the source of the repository. For this to work, Jenkinsfile must be loaded from either a multi-branch pipeline, or pipeline from SCM. Here, the agent is instantiated using the Dockerfile.

```
pipeline {
  agent { dockerfile true }
}
```

Option 3: The pipeline will execute the stage(s) using the container built on an agent using a custom Dockerfile sourced from SCM.

```
pipeline {
  agent {
      dockerfile {
          filename 'Dockerfile.hello-world'
          label 'preconfigured-node-to-download-this-
image'
      }
  }
}
```

Each option serves different purposes, and possesses advantages over others. While the first option is limited in the usage of args (it can be added before the script but can become cluttered and difficult to maintain), the second and the third options rely on the presence of Dockerfile as part of the repository that's hosting the source code.

It should be noted that all the three options are valid usage, and the pipeline will start with the agent setup with Docker args as shown in Figure 1.

Figure 1: Agent setup triggered by agent section

Jenksinfile, the pipeline script referred to earlier, is set up to pick any build agent that's capable of running Docker commands. In non-production environments, this kind of resource usage is not recommended and can lead to potentially unstable builds in the long run. It's a good practice to pin the pipeline to specific agent(s) that carry the label as given in the pipeline script.

While the dedicated build agent is capable of serving the build job request, as routed by the master, the other usage pattern is to spin-up build agent(s) (in the cloud or on-premise) on-demand as per the system configuration in the Jenkins master. The provision to create the build agent, when one is not available, greatly reduces the upfront investment on capacity planning but one should be aware that the provisioning duration, i.e., the agent creation time, will delay the actual start of the build until such an agent is online and can communicate with the master.

Hello World

Let us refer to a simple (public) image from the Docker Hub registry: *https://hub.docker.com/r/tutum/hello-world/*. Based on the description provided at the Docker Hub, this image is meant to test Docker deployments. It has Apache with a 'Hello World' page listening in on Port 80.

The rest of the article will focus on developing the pipeline script, gradually. However, the final and functional version can be accessed from the branch, the docker-build that hosts the script, in the following public GitHub repository: *https://github.com/mramanathan/jenkins_pipeline_demo/blob/be129179271b1b03417 27f93a399fb34d8133c6d/Jenkinsfile*.

And the associated Dockerfile (two lines of code) is quite basic, as shown in Figure 5. This is stored at the root of SCM, and is available in the same public GitHub repository:

https://github.com/mramanathan/jenkins_pipeline_demo/blob/1e725e0c98e59766fd1fc4fba3c98276146fb5e6/Dockerfile.

As noted earlier, the pipeline script refers to any available node, where the build will be scheduled by the Jenkins master.

Registry settings

On successful completion of validation, the pipeline script will push the image to the repository, *raspamdocker/osfy* which is hosted at the *https://registry.hub.docker.com*. The repository's name and URL will be set in the environment section of the pipeline script as shown in Figure 2. This global

```
6       environment {
7           IMAGE = "raspamdocker/osfy"
8           REGISTRY = "https://registry.hub.docker.com"
9       }
```

Figure 2: Values for the repository and the registry